
LECTURE NOTE

Lecture 5: Efficient nlp's

AI509 - Natural Language Processing

Simon Holm
Fall - 2026



Contents

1	Pre-processing	3
1.1	Practical problems	3
1.2	Mosaic Data Shard (MDS)	3
1.3	Innovate new data format (JINX)	3
1.4	BINX	4
2	Model architectures	5
2.1	Hierarchical reasoning models	5
2.2	Federatedly trained experts	5
2.2.1	Indirect communicating	5
2.3	Model weight editing	5
3	Distributed training	6
3.1	Practical problems	6
3.2	Decoupled Momentum (NousResearch):	6
3.3	FlexDeMo:	6
4	Model inference	6
4.1	Efficiency requirements	6
4.2	Prefill Decode Disaggregation	7
5	Quantization	7

1 Pre-processing

1.1 Practical problems

- Text datasets can cost a lot of storage. Sets like `Common Crawl` is measured in PB, and even filtered sets like `FineWeb` are tens of TB.
- Datasets might be multi-modal (like text + images), which is difficult to account for data format wise. While text itself is relatively easy to handle. Different combinations of data is difficult to standardize formats for.
- There are multiple ways to go about *streaming* data through the ram. Since you have to do it sequentially, the file format is a critical design choice for efficiency.
- if some operations require large chunks or even the whole dataset at once. At 1GB, this might be easy, but 50TB datasets are impossible to store in ram at once.

1.2 Mosaic Data Shard (MDS)

Storing each sample as a separate file is pretty inefficient since reading samples requires,

open file → read file → close file

MDS is specifically build for streaming sequentially efficiently by packing many samples in larger **shards** files. These shards use an **index**, keeping track where each sample starts (using byte offset) and how long it is.

1	<code>shard.00000.mds</code>	← samples 0–9999 back-to-back
2	<code>shard.00001.mds</code>	← samples 10000–19999
3	<code>index.json</code>	← shard list + per-sample offsets

This mean that one can acces any sample i at any point (*random acces*). This matters for shuffling in training. A shard like these is opened once, and many sample can be read from it.

1.3 Innovate new data format (JINX)

A problem with MDS is that its basicly impossible to inspect adn read from a human perspective, JSONL, is the perfect opposite being easy to read and inspect, but slow to acces. JINX keeps both advantages by storing data like JSONL, but keeping an index like MDS

1	<code>data.jinx</code>	← readable JSON records (open it, grep it, debug it)
2	<code>index</code>	← "sample 5,000,000 starts at byte 8,123,456"

While data is not stored in binary like MDS, data is stored on disc in bytes:

1	Text:	H	e	l	l	o
2	Bytes:	72	101	108	108	111

Additionally, JINX also uses lazy decoding, when decoding these bytes to readable text. That means only doing that work for the samples and fields you actually use, and only when you use them.

1.4 BINX

In some instances, data might be bad to store as text, in that case BINX, attaches a separate binary file. This is smart and handles issues with multi-modal data.



Figure 1: the secret sith lord 🕶️

For example, you could store text with the JINX and images with BINX

2 Model architectures

2.1 Hierarchical reasoning models

HRM's has two small recurrent models that run at different speeds.

- Low-level module (*fast*): Does fast local computations via update steps.
- High-level module (*slow*): Plans and steers globally the fast module each K *low-level* steps.

$$\text{HRM}(\cdot) \mathcal{H}^L \circ \mathcal{L}^L \circ \mathcal{L}^L \circ \mathcal{L}^L(\cdot)$$

2.2 Federatedly trained experts

Many organizations (companies) want to use models, which works on their private data. The issue is that many of this data is for different reasons something which they cannot share. Instead of training many different models (which is expensive), training a base model for each organization is much cheaper. Then each organization can train a specific expert within that model on their own data. This is much cheaper

A good example is FlexOlmo, where they did exactly this. MoE where each expert is finetuned **independently** from the same public base model.

2.2.1 Indirect communicating

Since multiple experts can be trained from the same latent space. While they drift away from the initial base model during finetuning, in many cases they are still compatible to some degree.

- Shared layers are frozen and act as an anchor, keeping experts compatible.
- Experts don't talk directly. They write to the shared hidden state, which later layers read.
- E.g. an English (domain) expert + a Danish (language) expert, so knowledge transfers across languages.

2.3 Model weight editing

Change a model by **editing its weights directly** might be better than training new model from scratch

This can be done by up-cycling a base model into a MoE model. Copying the FNN N times, so each one becomes a separate expert. Train a *router* to decide which experts each token gets sent to. This router can be modified when experts gets added/removed.

Multiple methods to deal with routers and experts. Experts can be merged after training for specific tasks. After a merge, a new expert might have features from both, previously separate experts.

3 Distributed training

3.1 Practical problems

In regular data-parallel training, every independent GPU optimizes the same model by storing a copy of it and computing gradient on different data, then averaging their gradient updates.

nodes	Bandwidth
Intra	63-1800 GB/s
Inter	0.125-100 GB/s

Table 1: This figure shows for intra (*inside* the model) and inter (*between* the model) nodes how much data a connection can move per second

Essentially, this means that while each machine can be super fast, having 100 GPUs would need multiple machine is a bottleneck due to the slower inter connections.

3.2 Decoupled Momentum (NousResearch):

DeMo handles this issue by sending far less data. Instead of each GPU sending its full gradients (which amounts to GB of data), only the most important parts of each GPU's accumulated gradients (*momentum*) are shared each step. The rest of the gradients is kept locally until it becomes important enough to send.

3.3 FlexDeMo:

While DeMo is much more efficient, it compresses gradients everywhere in the process. That means that even inside the machines (where data is transmitted fast) the gradients are still compressed as described above. FlexDeMo only applies this where it matters, so gradient are fully sent inside the machines (where it is fast), but compressed when sharing with other machines (where its expensive).

Analogy: *colleagues in the same building just talk and share everything, while offices only share one-line summaries to each other.*

4 Model inference

Model *inference* is usage of the model (like answering a prompt sent by a user).

4.1 Efficiency requirements

Big model (like ChatGPT) has a larger inference, since thousands of users are simultaneously sender prompts at the same time.

When users are waiting for the model to answer, speed is measured in two ways

- TTFT (Time To First Token): how long does it take until the first word appears (thinking delay)
- TPS (Tokens Per Second): how fast is the text stream.

While training a giant model can be expensive in itself, the inference has to run millions of times a day, which **dwarves** the cost of training.

4.2 Prefill Decode Disaggregation

Prefill and Decode are two phases of answering a prompt.

- **Prefill:** The model reads the whole prompt and processes token in parallel, which is *compute-bound* and thus determines **TTFT**
- **Decode:** The model generates each answer token one at a times, this is *memory-bound*, determining **TPS**.

5 Quantization

1.58-bit quantization (MSR) is a type of quantization (compress weights so weight can be represented in fewer bits) where weights can only be -1, 0 or +1 which equals $\log_2(3) = 1.58\text{bits}$